МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: М. С. Мастрюкова
Группа: М8О-304БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Реализовать «камеру» с возможностью её перемещения и освещение по модели Блинн-Фонга с использованием множества типов источников света.

Вариант №6: пирамида

Базовое задание: отрисовка пирамиды относительно камеры с единственным направленным источником освещения. Все параметры камеры, источника освещения и материала по модели Блинн-Фонга, а также рассеянный свет (ambient light) должны быть изменяемыми из пользовательского интерфейса ImGUI.

Дополнительные задания:

1. Разделить внешние данные на два набора дескрипторов: один для сцены (SceneUniforms + storage-буферы источников), другой для модели (ModelUniform с матрицей трансформации и параметрами материала).
2. Реализовать освещение по модели Блинн-Фонга массивом точечных источников света из storage-буфера, с затуханием по закону обратных квадратов. В UI - добавление и удаление источников.
3. Реализовать освещение по модели Блинн-Фонга массивом прожекторных источников света из storage-буфера, с затуханием по закону обратных квадратов и мягкими краями (внутренний/внешний углы).
4. Визуализация источников: куб с цветом источника для точечного и прожекторного источников; конус из белых линий для прожектора, показывающий направление и угол. Возможность показа/скрытия в UI. Фигуры визуализации не освещаются по модели Блинн-Фонга.

Метод решения

В работе использовался API Vulkan совместно с библиотеками GLFW (создание окна), ImGUI (интерфейс) и GLM (линейная алгебра). Код лабораторной работы №1 был расширен: добавлены управляемая камера, освещение по модели Блинн-Фонга и несколько типов источников света.

Матрица вида строится через glm::lookAt(position, position + front, up). Направление взгляда камеры определяется двумя углами Эйлера: yaw (рыскание, поворот вокруг оси Y) и pitch (тангаж, наклон вокруг оси X). Направление взгляда front вычисляется из углов через cos/sin. Положение

и ориентация камеры, а также угол обзора (FOV) задаются интерактивно через интерфейс ImGUI. Матрица проекции формируется либо через glm::perspective (перспективная с динамическим FOV), либо через glm::ortho (ортографическая). При работе с Vulkan учитываются две особенности: диапазон NDC по оси Z равен [0, 1], а инвертированное направление оси Y в NDC компенсируется умножением proj[1][1] на -1.

Для корректного расчёта освещения была сгенерирована геометрия октаэдра: для каждой из 8 граней создаются три вершины с общей нормалью, вычисленной как векторное произведение рёбер грани n = normalize(cross(p1 - p0, p2 - p0)). Это позволило процедурно задать перпендикулярные векторы нормалей для каждой грани, избегая нежелательного сглаживания освещения на острых рёбрах (flat shading). У вершины, помимо позиции и цвета, добавлен атрибут normal. В вершинном шейдере нормаль передаётся во фрагментный шейдер и трансформируется нормальной матрицей mat3(transpose(inverse(model))).

Данные разделены на два набора дескрипторов. Set 0 (данные сцены, общие для всех объектов): binding 0 — uniform-буфер (std140) с комбинированной матрицей View-Projection, мировой позицией камеры, параметрами фонового освещения (ambient), параметрами направленного источника (вектор направления, цвет, интенсивность) и количеством активных точечных и прожекторных источников; binding 1 — storage-буфер (std430) с динамическим массивом параметров точечных источников; binding 2 — storage-буфер (std430) с динамическим массивом параметров прожекторных источников. Set 1 (данные конкретной модели): binding 0 — uniform-буфер (std140) с матрицей трансформации модели (Model), коэффициентом диффузного цвета (альбедо Kd), цветом зеркального блика (Ks) и показателем степени блеска (shininess, alpha).

Расчёт освещения каждого пикселя выполняется во фрагментном шейдере в мировых координатах. Фоновое освещение (ambient) рассчитывается как произведение цвета окружающего света на базовый цвет поверхности Kd и равномерно освещает все грани. Диффузная составляющая по закону Ламберта пропорциональна косинусу угла между нормалью поверхности n и направлением на источник света l: I_diff = Kd * light_color * max(dot(n, l), 0.0). Зеркальная составляющая по Блинн-Фонгу рассчитывается через единичный вектор половинного угла h = normalize(l + v), где v — вектор направления на камеру: I_spec = Ks * light_color *

pow(max(dot(n, h), 0.0), alpha). Итоговый цвет: final_color = ambient + diffuse + specular.

Затухание света с расстоянием для точечных и прожекторных источников реализовано по закону обратных квадратов: attenuation = intensity / (d^2 + epsilon). Мягкие края прожектора вычисляются на основе косинуса угла между направлением луча к поверхности и центральной осью прожектора: spot_factor = smoothstep(cos_outer, cos_inner, cos_theta).

Направленный источник хранится в SceneUniforms (set 0, binding 0) как направление и цвет с интенсивностью. Он не имеет позиции, все лучи параллельны, вектор L одинаков для всех фрагментов. Точечные источники хранятся в storage-буфере (set 0, binding 1). Для каждого источника во фрагментном шейдере вычисляется вектор L, затухание по закону обратных квадратов, диффузная и зеркальная составляющие. В UI реализованы кнопки Add/Remove и слайдеры позиции, цвета и интенсивности. Прожекторные источники хранятся в storage-буфере (set 0, binding 2). Для каждого вычисляется угол theta = dot(-L, direction). Мягкие края реализованы через плавный переход между внутренним и внешним углами конуса, затухание — по закону обратных квадратов. В UI — Add/Remove, слайдеры позиции, направления, цвета, интенсивности и углов (внутренний, внешний).

Для визуального отображения источников света были созданы отдельные unlit-пайплайны с шейдерами без расчёта освещения (unlit.vert / unlit.frag) и собственным VkPipelineLayout с push-константами. Передача матриц положения маркеров и их цветов осуществляется через механизм push constants, поэтому фигуры визуализации не участвуют в расчёте Блинн-Фонга — они просто рисуются отдельным конвейером. Всего в коде используется три графических пайплайна: vk_pipeline для освещённого октаэдра (topology TRIANGLE_LIST, основной Blinn-Phong layout), vk_unlit_pipeline для кубов-маркеров источников (topology TRIANGLE_LIST, unlit layout) и vk_unlit_lines_pipeline для конусов прожекторов (topology LINE_LIST, unlit layout). Точечные и прожекторные источники отображаются яркими кубами постоянного цвета источника. Для прожекторов дополнительно строится один конус из белых линий по внешнему углу (outer_angle_deg), показывающий направление и форму светового пучка. Возможность показа/скрытия визуализации — галочка в UI (show_light_visualizers).

Используемые объекты Vulkan:

1. VkBuffer, VmaAllocation — вершинный, индексный, uniform- и storage-буферы;
2. VkDescriptorSetLayout, VkDescriptorPool, VkDescriptorSet — описание и привязка буферов к шейдерам;
3. VkPipelineLayout, VkPipeline — конфигурация графических конвейеров (три пайплайна: освещённый, unlit-fill для маркеров и unlit-lines для конусов);
4. VkRenderPass, VkFramebuffer, VkCommandBuffer — организация прохода рендеринга и запись команд;
5. VkShaderModule — загрузка скомпилированных шейдеров;
6. ImGUI — графический интерфейс;
7. GLFW — создание окна и ввод.

Результаты

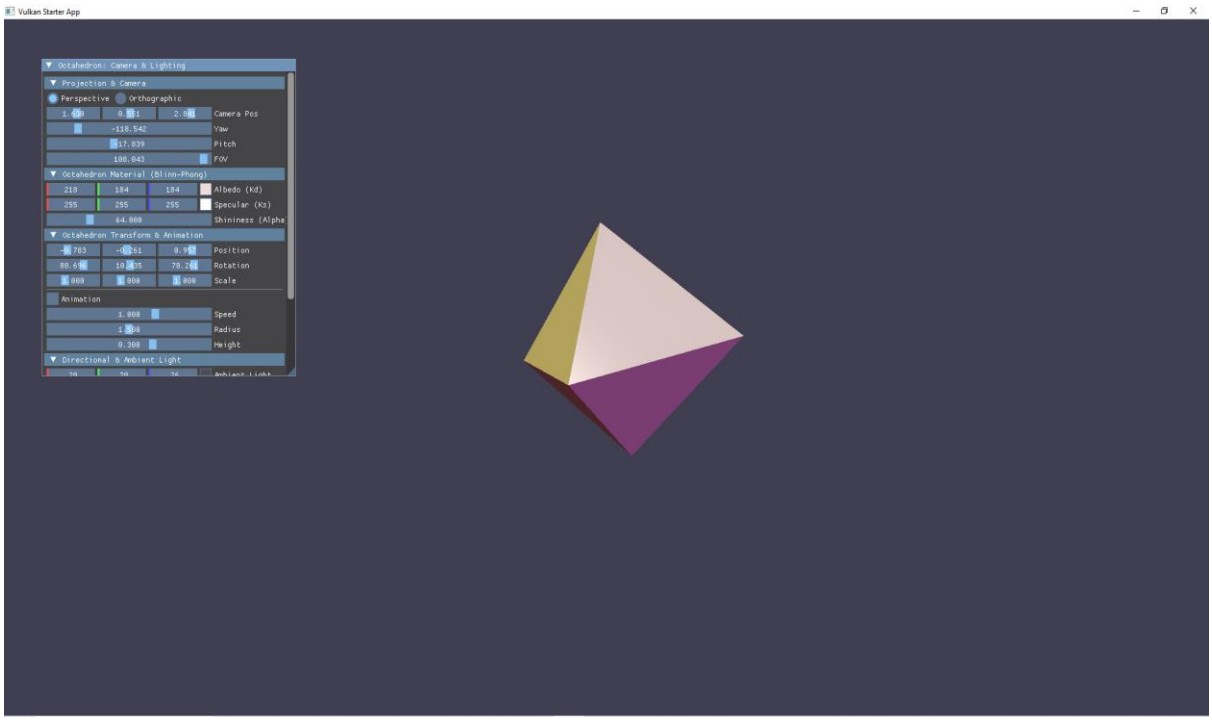


Рис. 1. Общий вид сцены: октаэдр, освещённый направленным источником света

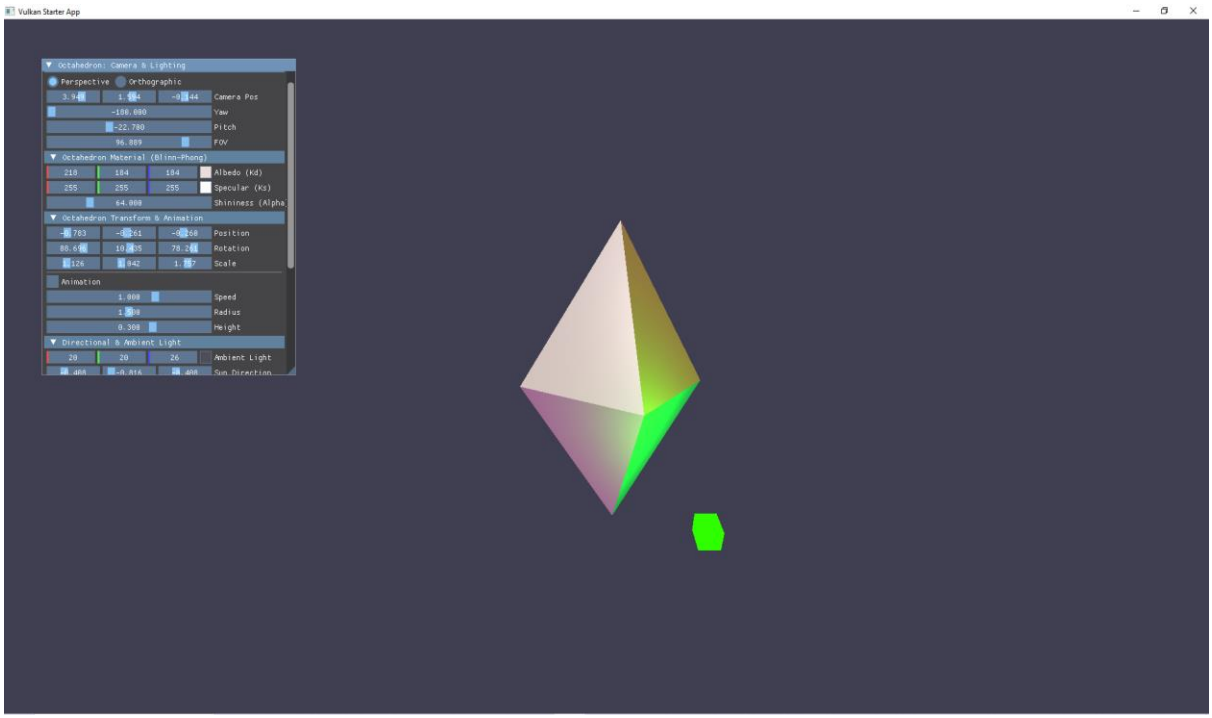


Рис. 2. Освещение с точечным источником: виден яркий куб и затухание света по закону обратных квадратов

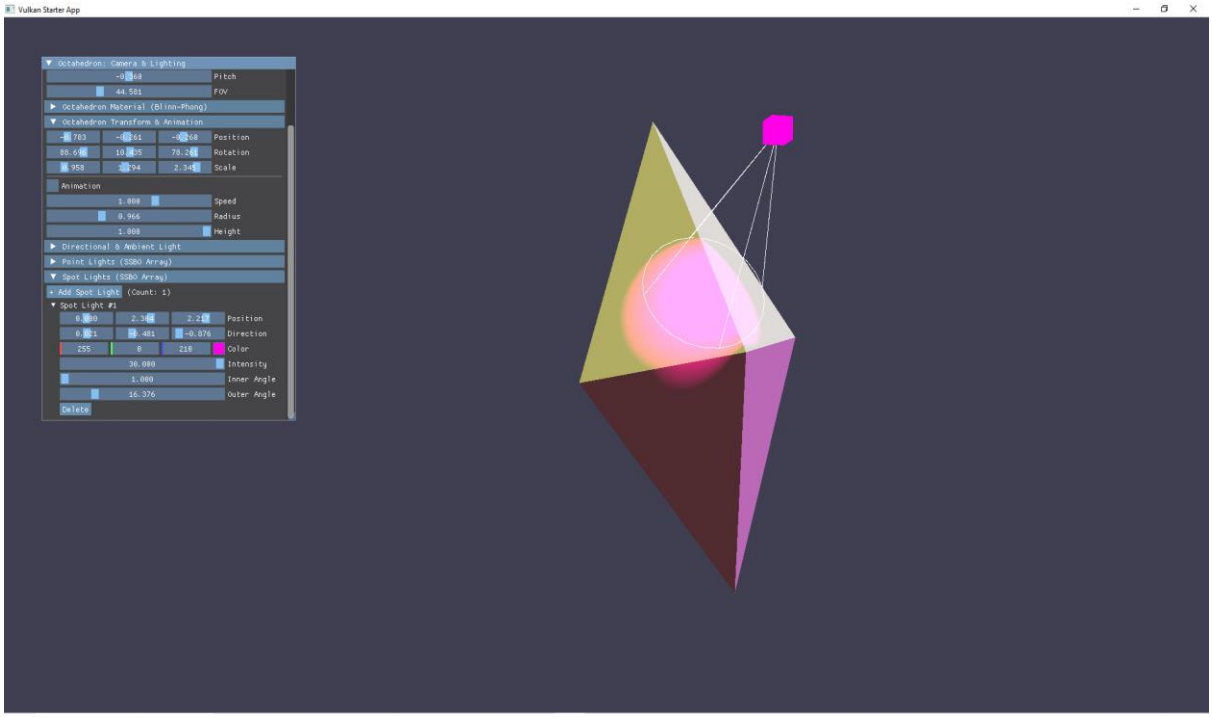


Рис. 3. Прожекторный источник: конус из белых линий показывает направление и угол

Выводы

В ходе лабораторной работы я реализовала управляемую камеру (yaw/pitch с расчётом направления взгляда и построением видовой матрицы через glm::lookAt) и динамическое освещение по модели Блинн-Фонга с использованием нескольких типов источников света. Я освоила модель освещения Блинн-Фонга: вычисление диффузной, зеркальной и ambient-составляющих во фрагментном шейдере, половинчатый вектор h = normalize(l + v) и работу с нормалями вершин. Научилась генерировать нормали процедурно через векторное произведение рёбер каждой грани и трансформировать их нормальной матрицей mat3(transpose(inverse(model))). Отдельно я разобралась с использованием storage-буферов для массивов источников света переменной длины: количество активных ламп передаётся в SceneUniforms, а сами данные лежат в SSBO, что позволяет обойтись без жёстко заданного в шейдере максимума. Для точечных и прожекторных источников реализовано затухание по закону обратных квадратов, а для прожекторов — мягкие края через smoothstep по косинусу угла.

Также я научилась визуализировать источники освещения: кубы с цветом источника для точечных и прожекторных ламп и конус из белых линий (LINE_LIST) для прожекторов. Визуализация вынесена в отдельные unlit-пайплайны с push-константами, поэтому фигуры-маркеры не участвуют в расчёте освещения. Реализовала возможность показа/скрытия визуализации через UI. Все задания лабораторной работы, включая четыре дополнительных, выполнены полностью: программа работает в реальном времени, поддерживает динамическую смену проекции (перспективная/ортографическая), трансформации объектов, освещение несколькими источниками с редактированием параметров в UI, визуализацию источников с возможностью показа/скрытия.